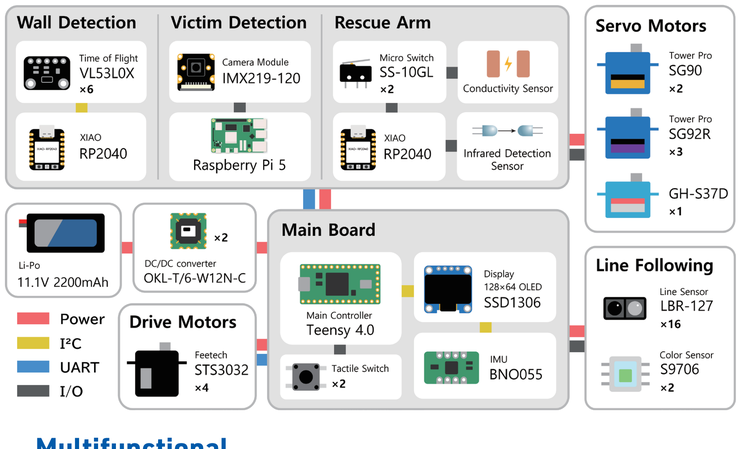
Hardware

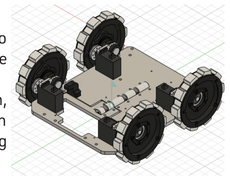
Multi-Processor Architecture

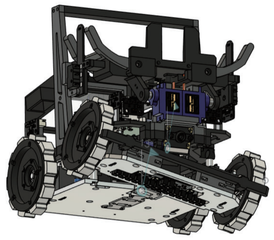
The robot uses a Teensy 4.0 as the main controller, a Raspberry Pi 5 for image processing and AI inference, and two XIAO RP2040 boards as sub-controllers. The Raspberry Pi processes camera data, while the RP2040 boards handle dedicated sensor and actuator tasks. This distributed architecture enables high-speed control, accurate vision-based recognition, and reliable parallel processing.



Powerful Power Train

The robot is driven by four STS3032 serial servo motors. These compact yet powerful motors enable the robot to overcome bumps reliably.
The wheels feature a large diameter, narrow width, and custom-molded silicone tires. This design provides high grip on bumps and ramps, improving traction and driving performance.



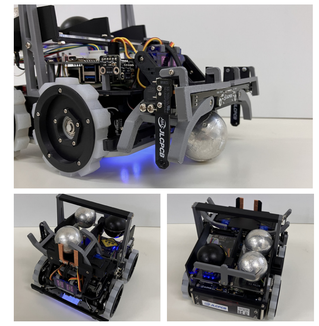


Twisting Mechanism

The front and rear halves of the robot are connected by linear shafts, allowing them to twist relative to each other. This mechanism keeps all wheels in contact with the floor on uneven terrain, improving driving performance on bumps and ramps. It also maintains a stable distance between the line sensors and the floor, enabling smooth and reliable line following even at transitions between flat tiles and ramps.

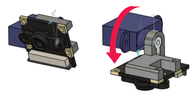
Multifunctional Rescue Mechanism

Our rescue mechanism can collect, identify, classify, store, and evacuate victims within a compact system. By storing all victims simultaneously, the robot reduces travel time and improves rescue efficiency. The rescue arm uses sensors to detect obstacles, confirm victim capture, and identify victim conductivity. A one-sided gripper and tilting gate mechanism enable victim sorting and selective evacuation.



Rotating Camera Mechanism

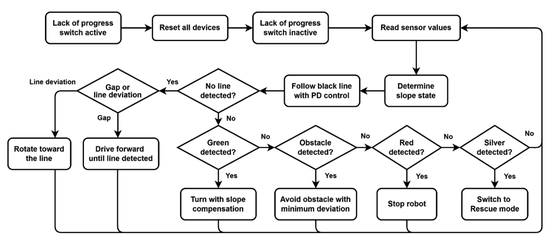
The camera is mounted on a servo-driven mechanism and is primarily used for victim detection. In gap situations, the camera rotates downward and is used for camera-assisted gap control.



Software

Line Following

The robot calculates the line position from line sensor data and follows the line using PD control. When it detects an intersection with the color sensor, it determines the appropriate turn based on the elapsed time since the last black line was detected. On ramps, it applies slope-specific compensation to optimize line-following performance for each incline. When no line is detected, it first moves backward and determines whether the situation is a gap or a line following error, significantly reducing the risk of LoP. When encountering an obstacle, it uses distance and touch sensors to navigate around it while maintaining the shortest possible clearance.

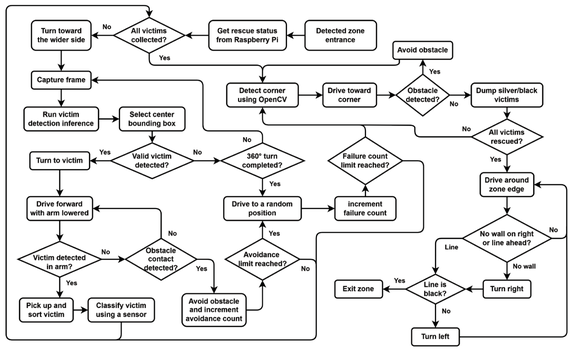


Camera-Assisted Gap Control

Line sensors provide stable and low-noise line following. However, because they can only detect the presence of a line, they cannot determine the direction of the line while traversing a gap.
To address this limitation, the robot uses a camera during gap traversal. When a gap is detected, a servo motor rotates the camera downward, and the line angle is calculated using OpenCV's PCACompute function. The robot then uses this angle to align itself with the recovery line.
By combining sensor-based line following with camera-based directional analysis, the robot can navigate challenging gaps more reliably.

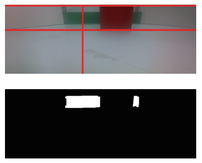
Evacuation Zone

An overview of the evacuation zone process is shown in the figure below. Throughout the rescue operation, the robot utilizes its multifunctional rescue mechanism and distance sensors to handle challenging environments with many obstacles. Because the robot collects victims regardless of their type, it can perform rescue operations efficiently without prioritizing specific victims. The number of rescued victims is stored on the Raspberry Pi throughout a run. After a LoP, the robot refers to this information, allowing it to perform optimally even on subsequent attempts. The algorithm is designed to handle various situations, such as failing to collect all victims or being unable to locate a corner.



Corner Detection

The robot detects the evacuation point using OpenCV-based image processing. Red and green regions are extracted from the camera image, and connected-component labeling is used to remove noise. The evacuation point is then identified based on the aspect ratio of the detected region. To improve robustness when obstacles partially occlude the evacuation point, the robot evaluates the two largest labeled regions instead of only the largest one.

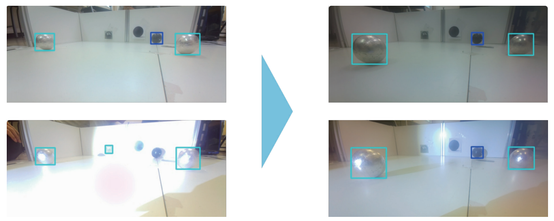


Victim Detection

We previously used an OpenCV-based algorithm for victim detection. However, distinguishing victims from the background was difficult, so we adopted YOLOv8, a deep learning-based object detection model. Using transfer learning based on the pretrained YOLOv8n model, the robot can detect both the position and type of victims from camera images.
We collected and processed our own dataset of approximately 3,000 images and manually annotated all victims. To improve robustness, the dataset included dummy balls, unlabeled images, and Blender-generated images. Additional datasets containing flashlight interference were also created to address the new competition rules.

Innovation

Although the victim detection model was trained using images containing flashlight interference, false detections still occurred under certain light intensities and angles.
To address this issue, we installed a polarizing film in front of the camera lens. This modification significantly reduced strong reflections from the floor and improved detection reliability under challenging lighting conditions.



TOKAI Ghost

League : **Rescue Line**
Country : **Japan**

OUR TEAM

Our team, TOKAI Ghost, consists of students from the same high school. We began participating in RoboCup Junior Rescue Line last season. Building on the experience we gained at RoboCup 2025 Salvador, we developed a powerful, compact, and reliable robot.

TAKUMI TAKANO
Captain, Hardware and Software Developer

Takumi Takano designed all the electronic components and most of the robot's structure, and he manages the development schedule. He also coded the algorithms for line following and rescue. Additionally, he coordinates the opinions of team members.

HONGYI DAI
Software Developer

Hongyi Dai coded the algorithms for detecting evacuation points. He also developed a camera-based assist system for line following. Furthermore, he is responsible for setting up a Raspberry Pi 5 environment and configuring the camera.

SHINICHI KADO
Software Developer

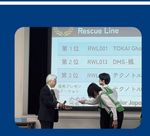
Shinichi Kado joined the team this season. He developed a machine learning model for victim detection. He also developed efficient methods for capturing images and augmenting data for the dataset. Thanks to his work in introducing new technology, the robot can reliably detect victims.

RoboCup Junior

JapanOpen 2026
- 1st Place • Software Award • IRS Award

RoboCup 2025 Salvador
- Rescue Line 5th Place
- Outstanding Design Award

FLL **FIRST 2025 Championship**
- Engineering Excellence Award Winner
- Encore Celebration Alliance FInalist



Supported by JLCPCB

X
note
YouTube